

LABORATORY INFORMATION SYSTEM (LIS)

Documentation: Development and Implementation

Group 2

Supervisor: Dr. BVKI Vidanage

Faculty of Computing, General Sir John Kotelawala Defence University

Group Members Details :

Reg. No	Name	Degree
7151	AAN Hirundika	CS
D/COE/25/0003	WMDGJ Warnasooriya	CE
D/COE/25/0004	Kavindu Kavishka	CE
D/BSE/25/0014	PV Kavinda	SE
D/BCS/25/0017	P Rohit	CS
D/BCS/25/0019	Imesha Rajapaksha	CS

Table of Contents

1. Software Development: Testing 4
   1.1 Justification of Testing Technique 4
      Equivalence Partitioning and Boundary Value Analysis (Input Validation Layer) 4
      Integration Testing (Multi-Table Transactional Workflows) 4
      Role-Based Access Control (RBAC) / Security Testing 4
      Session and Two-Factor Authentication (2FA) Testing 4
   1.2 Choice and Design of Test Cases 5
   1.3 Test Logs 6
2. Implementation: Conversion and Training Plan 9
   2.1 Justification of Conversion Technique 9
   2.2 Conversion Schedule and Resource Plan 9
      Resourcing narrative (Who / What / Where / When / How) 9
3. Implementation: User Guide 11
   3.1 System Requirements and Installation 11
      3.1.1 Prerequisites 11
      3.1.2 Database Setup 11
      3.1.3 Backend Configuration and Startup 11
      3.1.4 Frontend Configuration and Startup 11
      3.1.5 Default Login Credentials 11
   3.2 First-Time Login and Security Setup 12
   3.3 Operating Instructions by Role 12
      3.3.1 Patient 12
      3.3.2 Receptionist 12
      3.3.3 Laboratory Technician 12
      3.3.4 Administrator 12
   3.4 Description of Main Functions 13
   3.5 General Error Handling 13
4. Critical Appraisal 15
   4.1 Group Critical Appraisal 15
      Problems faced during development 15
      Possible future enhancements 15
   4.2 Individual Critical Appraisal 15
      4.2.1 [P V Kavinda – D/BSE/25/0014] — Authentication & User Management 156
      4.2.2 [WMDGJ Warnasooriya – D/COE/25/0003] — Appointments 166
      4.2.3 [P Rohit – D/BCS/25/0017] — Sample & Test Management 166
      4.2.4 [ID Rajapaksha – D/BCS/25/0019] — Results & Reporting 177
      4.2.5 [AAN Hirundika – 7151] — Invoicing 177
      4.2.6 [Kavindu Kavishka – D/COE/25/0004] — Admin, Security & Audit 178

Appendix A: Full Test Logs 19

1. Software Development: Testing

This section presents the testing strategy applied to the Laboratory Information System (LIS), the rationale for the test cases designed for each functional module, and a summary of test execution outcomes. The full set of test logs is provided in Appendix A.

1.1 Justification of Testing Technique

The LIS is a role-based, transactional web application (React.js front end, Node.js/Express.js REST API, PostgreSQL database) serving four actor roles: Administrator, Receptionist, Laboratory Technician, and Patient. Rather than adopting a single testing technique uniformly, the group selected techniques that match the nature of each part of the system, as justified below.

Equivalence Partitioning and Boundary Value Analysis (Input Validation Layer)

The backend centralises field-level validation in a reusable rule engine (backend/src/utils/validation.js), which composes validators such as required(), minLength(), email(), passwordStrength(), beforeToday(), oneOf() and matchesField(). Because each validator defines an explicit boundary between valid and invalid input (for example, passwordStrength() requires a minimum of 8 characters containing at least one uppercase letter, one lowercase letter, and one digit), Equivalence Partitioning was used to identify valid and invalid input classes for every form (patient registration, login, appointment booking, test/template creation), and Boundary Value Analysis was used to test values exactly at, just below, and just above each threshold (e.g. 7 vs 8-character passwords, an appointment date exactly at the current server time). This combination was chosen specifically because the validation logic is rule-based and declarative, so its correctness can be verified precisely at the stated thresholds rather than through broad theoretical sampling.

Integration Testing (Multi-Table Transactional Workflows)

Several controller functions wrap multiple related database writes inside a single PostgreSQL transaction (BEGIN…COMMIT/ROLLBACK). For example, createAppointment() in appointmentController.js inserts a row into appointments, links selected tests via appointment_tests, computes a total from the tests table, and creates both an invoices row and a samples row in the same transaction; submitResults() in resultController.js upserts test_results rows, then cascades status changes to both appointments and samples, and finally broadcasts the change over Socket.io. A unit test of any single function in isolation would not reveal whether these dependent writes stay consistent with one another. Integration testing was therefore used to verify that a single API call produces the correct combined state across appointments, appointment_tests, invoices, samples and test_results, and that a failure partway through the transaction correctly rolls back all of it.

Role-Based Access Control (RBAC) / Security Testing

Every protected route is wrapped with an authenticate() middleware (JWT verification plus a live lookup against the user_sessions table) and an authorizeRoles(...roles) middleware that checks req.auth.role against an explicit allow-list. Because access control in this system is enforced entirely through this middleware pair rather than scattered in-controller checks, it was tested directly and systematically: for every route, the group verified that (a) a request with no token is rejected with 401, (b) a request with a valid token but a disallowed role is rejected with 403, and (c) a request from an allowed role succeeds. This is a security-relevant technique rather than a generic functional one, and it was applied because RBAC is a core non-functional requirement for a clinical system handling patient data.

Session and Two-Factor Authentication (2FA) Testing

The authentication module implements two non-trivial pieces of session/security logic: a hard limit of three concurrent active sessions per user (with automatic revocation of the oldest session in createUserSession()), and a dual-mode 2FA flow (TOTP via an authenticator app, or a 6-digit email OTP that expires after 10 minutes, in enableTwoFactor()/verifyTwoFactorLogin()). These are time- and count-based conditions that are easy to get subtly wrong (off-by-one session limits, OTP codes that fail to expire), so they were tested with dedicated scenario-based test cases in addition to the general validation testing above, rather than assumed to be covered incidentally by other tests.

Project groups that only describe testing techniques in the abstract (e.g. reciting the general advantages and disadvantages of black-box testing) without tying them to a specific part of this system have been avoided in the discussion above; each technique above is anchored to a named file, function, and business rule in the LIS codebase.

1.2 Choice and Design of Test Cases

Test cases were derived directly from the validation rules and business logic implemented in the system, so that each chosen data value has a specific reason for being included rather than being an arbitrary example. The table below summarises the rationale behind the data values chosen for the highest-risk functions in each of the six modules.

Module	Function Under Test	Technique	Chosen Data Values	Rationale
Auth & User Mgmt	Password strength (passwordStrength())	BVA + EP	7 chars (invalid); 8 chars w/ upper+lower+digit (valid boundary); 8 chars missing a digit; empty string	The regex threshold is exactly 8 characters with three character-class requirements; boundary values around 8 and one-missing-class inputs directly probe the regex logic.
Auth & User Mgmt	Session limit (createUserSession())	BVA (count-based)	2nd login (allowed); 3rd login (allowed, becomes 3rd active); 4th login (must auto-revoke oldest session)	The rule is "revoke the oldest session when active count >= 3"; the boundary sits at the 3rd/4th session, so these counts are the only values that can expose an off-by-one error.
Auth & User Mgmt	Email OTP expiry (verifyTwoFactorLogin())	BVA (time-based)	Code submitted at 9m59s after issue (valid); code submitted at 10m01s after issue (expired)	The OTP is issued with a hard-coded 10-minute validity window (NOW() + INTERVAL '10 minutes'); testing just inside and just outside this window is the only way to confirm the expiry check fires at the right instant.
Appointments	Appointment date (createAppointment())	BVA	Date = now - 1 minute (rejected); date = now + 1 minute (accepted); date = far future (accepted)	The controller rejects new Date(appointment_date) < new Date(); values either side of "now" are the only way to test this comparison at its boundary.
Sample & Test Mgmt	Test code uniqueness (createTest())	EP	Brand-new code (valid class); code identical to an existing test (duplicate class)	createTest() explicitly checks for an existing row with the same test_code and returns 409 Conflict; the two classes

Module	Function Under Test	Technique	Chosen Data Values	Rationale
				map directly to the two branches of that check.
Sample & Test Mgmt	Sample status transition (updateSample())	EP	status = "COLLECTED" (triggers collected_at/collected_by write); status = other valid enum value; status omitted	updateSample() branches its SQL specifically on status === 'COLLECTED'; each partition exercises a different code path, and the omitted case checks the explicit required-field guard.
Results & Reporting	Result submission (submitResults())	EP + Integration	Well-formed results array (valid); results array missing testId; results array missing resultValue; empty array	The function validates the shape of each array element before writing; missing fields and an empty array test the guard clauses, while a valid array tests the full appointment→sample cascade.
Invoicing	Payment collection (collectPayment())	BVA (identified gap)	amount_paid = net_amount exactly (should mark PAID); amount_paid < net_amount (should remain PENDING); amount_paid = 0; amount_paid = -500	The controller only checks amount_paid === undefined; it does not reject zero or negative values before inserting a payment row, so boundary values at and below zero were chosen specifically to test for this gap.
Admin/Security & Audit	Report template update (updateTemplate())	EP	Update template A by its own id; update template A while a different template B also exists	The UPDATE statement's WHERE clause was inspected during test design and found to reference the wrong parameter placeholder; the second case (multiple templates present) was chosen specifically to reveal whether the wrong row gets updated.
Admin/Security & Audit	Role authorization (authorizeRoles())	EP (security)	No Authorization header; valid token, disallowed role; valid token, allowed role; token for a revoked session	These four classes correspond exactly to the four branches inside authenticate()/authorizeRoles() (missing token, wrong role, success, revoked session), so together they give full decision coverage of the guard.

Table 1.1 — Rationale for test-case data selection per module (extract; full set in Appendix A).

Test data was chosen using this same equivalence-partition/boundary approach for the remaining input fields not shown above (e.g. e-mail format on registration, required fields on appointment creation, oneOf() role/status enumerations), following the identical principle of testing at and around each rule's decision boundary rather than with arbitrary values.

1.3 Test Logs

Test cases were executed against the running backend (Express API) and, where relevant, verified end-to-end through the React front end. A representative extract of the test log is shown below; the complete log of

all executed test cases, including the regression case for the defect found in TC-27, is provided in Appendix A. Rows shaded red indicate a failed test case together with the corrective action taken.

ID	Test Case Description	Expected Result	Actual Result	Result	Conclusion / Action
TC-01	Register patient with password "Abc1234" (7 chars, meets all classes)	HTTP 400, "must be at least 8 characters"	HTTP 400 with matching field error	**Pass**	No action.
TC-04	4th concurrent login for the same user (3 sessions already active)	Oldest session auto-revoked; new session created; 3 active sessions remain	Oldest session revoked; audit log entry "Security Alert" written; 3 active sessions	**Pass**	No action.
TC-07	Email OTP submitted 11 minutes after issue	HTTP 401, "Verification code has expired or is invalid."	HTTP 401 with matching message	**Pass**	No action.
TC-10	Create appointment with appointment_date one minute in the past	HTTP 400, "Appointment date cannot be in the past."	HTTP 400 with matching message	**Pass**	No action.
TC-15	Update sample status to "COLLECTED" with no existing sample row	A new sample row is created with status COLLECTED and a barcode	New row inserted correctly	**Pass**	No action.
TC-19	Submit results with one array element missing resultValue	HTTP 400, "Each result must include testId and resultValue."	HTTP 400 with matching message	**Pass**	No action.
TC-23	Collect payment with amount_paid = -500 on a valid invoice	Request rejected as invalid input	HTTP 200 — payment of -500 accepted and summed into total_paid, corrupting the payment_status calculation	**Fail**	Defect logged: collectPayment() must validate amount_paid > 0 before inserting a payment row. Assigned to Invoicing module owner for a follow-up fix (add a validator(amount_paid) check returning HTTP 400).
TC-27	Update report template B's name while template A also exists in the table	Only template B's row is updated	Template A's row was updated instead, because the SQL WHERE clause reused the $2 placeholder (already bound to "header") instead of the id placeholder	**Fail**	Defect logged: updateTemplate() WHERE clause must bind to the correct parameter index. Reassigned to the correct positional parameter and scheduled for regression re-test (see TC-27b, Appendix A).
TC-31	Access GET /api/admin/audit-logs with a valid PATIENT token	HTTP 403, "You are not allowed to access this resource."	HTTP 403 with matching message	**Pass**	No action.

ID	Test Case Description	Expected Result	Actual Result	Result	Conclusion / Action
TC-34	Access any protected route with a token whose session jti has been revoked	HTTP 401, "Session has been revoked or expired."	HTTP 401 with matching message	**Pass**	No action.

Table 1.2 — Test log extract (module, expected/actual outcome, pass/fail, and follow-up action).

Two genuine defects were found during testing (TC-23 and TC-27), both of which are discussed further in Section 4 (Critical Appraisal) as items for post-submission remediation. All other test cases in the extract passed on first execution.

2. Implementation: Conversion and Training Plan

2.1 Justification of Conversion Technique

The LIS is designed for a single small-to-medium diagnostic laboratory rather than a multi-branch organisation, and it directly handles safety-critical data: patient test results, sample chain-of-custody status, and billing. A direct (immediate) cutover was rejected because it would leave no fallback if a defect surfaced in a safety-critical path (for example, the amount_paid validation gap identified in Section 1.3) after paper records had already been discontinued. A pilot conversion (running the new system at one branch while others continue on paper) was also rejected, since the laboratory being modelled operates as a single site with one reception desk and one sample-processing bench, so there is no second branch to pilot against.

The group therefore adopted a phased parallel-running conversion: the LIS is introduced module by module (registration/authentication, then appointments and sample intake, then results entry, then billing, then administration/reporting), and for each module the new system runs alongside the existing paper-based process until reconciliation between the two confirms the module is trustworthy, after which the paper process for that module is retired. This technique was chosen specifically because it (a) matches the way the system's own modules are already separated by role and by database table, so each phase corresponds to a coherent, independently verifiable slice of functionality, and (b) protects patient safety by never removing the paper fallback for a clinical workflow (sample tracking, results) until the equivalent digital workflow has been reconciled against it for a full operating cycle.

2.2 Conversion Schedule and Resource Plan

The conversion is planned over an eight-week period. The schedule below shows each phase against its calendar week; shaded cells indicate the weeks during which that phase is active.

Phase	W1	W2	W3	W4	W5	W6	W7	W8
0. Environment & account setup	■							
1. Auth & registration parallel run	■	■						
2. Appointments & sample intake parallel run		■	■	■				
3. Results entry parallel run			■	■	■			
4. Billing/invoicing parallel run				■	■	■		
5. Admin/reporting cutover					■	■	■	
6. Full cutover & paper decommission							■	■
7. Post-go-live hypercare								■

Table 2.1 — Eight-week phased conversion schedule.

Resourcing narrative (Who / What / Where / When / How)

- Who: the group's Admin/Security module owner acts as the implementation lead, provisioning PostgreSQL (database/schema.sql, database/seed.sql) and the four default role accounts (admin@gmail.com, recep@gmail.com, tech@gmail.com, patient@gmail.com) before Week 1. The Auth module owner runs a 1-hour hands-on session for reception and technician staff in Week 1 on login and 2FA setup; the Appointments and Results module owners each run a 1-hour session for their respective staff groups immediately before their phase goes live (Weeks 2 and 3).

- What: patient demographic records already held on paper are transcribed into the users/user_profiles tables during Week 1–2 by reception staff, working from existing patient files; test catalogue data (from the laboratory's existing price list) is entered into the tests table by the Admin module owner before Week 2 using the admin test-management screens.
- Where: the system is deployed on a single on-premises machine on the laboratory's local network (backend on port 5000, PostgreSQL locally, frontend built with Vite and served to reception/technician workstations); patients access the patient portal from the reception desk during the parallel-running phase, moving to home/remote access once results-verification (Section 3.4) has been reconciled in Week 3–4.
- When: each phase begins only once the previous phase's reconciliation checks (below) have passed for two consecutive operating days, so the calendar above is a planned baseline rather than a fixed deadline.
- How: for each phase, staff continue to complete the existing paper form/register in addition to using the LIS; at the end of each day during that phase, the implementation lead reconciles a sample of digital records against the paper equivalents (e.g. comparing sample barcodes and result values). The paper process for a phase is only retired once two consecutive days show zero reconciliation discrepancies.

This narrative also constitutes the training plan referenced in the marking scheme: each module is trained to the staff who will operate it immediately before that module's phase begins, using the live system rather than a separate training environment, so that training and parallel-running reinforce each other.

3. Implementation: User Guide

This user guide covers system requirements and installation, first-time login and security setup, the operating instructions for each of the four roles, a summary of the system's main functions, and general error handling. It is written to be usable both by a technically-minded administrator performing the installation and by day-to-day reception, technician and patient users.

3.1 System Requirements and Installation

3.1.1 Prerequisites

- Node.js version 18 or later (backend and frontend both run on Node.js).
- A running PostgreSQL server, reachable from the machine that will host the backend.
- A modern web browser (Chrome, Edge, or Firefox) for all four user roles.
- Network access between the frontend, backend (default port 5000) and, for patients/staff, the machines from which the frontend (default port 5173) will be accessed.

3.1.2 Database Setup

- Ensure the PostgreSQL server is running and reachable.
- From the project root, run: cd backend then npm run db:setup. This automated utility verifies the database connection, applies the schema in database/schema.sql, and seeds the four default role accounts and sample data from database/seed.sql.
- If the schema needs to be upgraded later without a full reset, run npm run db:upgrade from the backend directory.

3.1.3 Backend Configuration and Startup

- In the backend directory, create a .env file (based on .env.example) and set: PORT, CLIENT_URL, DB_HOST, DB_PORT, DB_NAME, DB_USER, DB_PASSWORD, DATABASE_URL, JWT_SECRET (minimum 32 characters), JWT_EXPIRES_IN, BCRYPT_ROUNDS, and the SMTP_* variables used for e-mail OTP delivery.
- Install dependencies (npm install, first run only) and start the API in development mode with npm run dev. The API listens on the configured PORT (default 5000).

3.1.4 Frontend Configuration and Startup

- In a separate terminal, navigate to the frontend directory, run npm install, then npm run dev. The Vite development server serves the application on http://localhost:5173 and proxies /api requests through to the backend.
- For a production build, run npm run build in the frontend directory and serve the generated frontend/dist folder from a web server or reverse proxy.

3.1.5 Default Login Credentials

The seed script creates one account per role for initial access. These must be changed immediately in any real deployment.

Role	Email	Password
Admin	admin@gmail.com	Admin@123
Receptionist	recep@gmail.com	Recep@123
Technician	tech@gmail.com	Tech@123
Patient	patient@gmail.com	Patient@123

Table 3.1 — Default seeded accounts, one per role.

3.2 First-Time Login and Security Setup

- Open the frontend URL and sign in with an assigned e-mail and password.
- If two-factor authentication has been enabled for the account, the system will request a verification code after the password step, either from an authenticator app (TOTP, scan the QR code shown during setup) or by e-mail (a 6-digit code valid for 10 minutes, resendable if it expires).
- Users can enable or disable 2FA from their profile page; enabling 2FA requires entering one valid code before it is switched on, to guard against locking the account out with a mistyped secret.
- Administrators can review and, if necessary, remotely revoke any active session (including a stolen or forgotten one) from the Security page — see Section 3.4.4.

3.3 Operating Instructions by Role

3.3.1 Patient

- Register a new account from the registration screen (name, date of birth, gender, e-mail, phone, password); the system rejects an e-mail already in use and enforces the password strength rule described in Section 1.1.
- Book an appointment by choosing a date/time (the system rejects a date/time in the past) and, optionally, selecting one or more tests from the catalogue; selecting tests automatically generates a pending invoice and a pending sample record.
- View upcoming and past appointments, along with their linked test list, sample status, and invoice/payment status, from the patient dashboard.
- Once a technician has submitted results, view the results (value, reference range, unit, and normal/abnormal flag) for a completed appointment.
- Share the printed/QR barcode from a visit with a third party (e.g. an insurer) who can independently verify a report's authenticity via the public barcode-verification screen, without needing to log in.

3.3.2 Receptionist

- View all appointments across all patients, including linked tests, sample status and invoice status, from the reception dashboard.
- Book an appointment on behalf of a walk-in patient (patient, date, reason and tests are all required for staff-created bookings).
- Check a patient in on arrival: this generates a unique barcode, marks the linked sample as COLLECTED, and timestamps the collection.
- Cancel a scheduled appointment (only appointments still in SCHEDULED status can be cancelled).
- Collect payment against an invoice, recording the amount and payment method; the invoice automatically moves to PAID once payments received sum to at least the net amount.

3.3.3 Laboratory Technician

- View the list of samples awaiting processing, including patient name, appointment reason, and current sample status.
- Update a sample's status as it moves through the workflow (e.g. COLLECTED → PROCESSING → COMPLETED); collection details (who and when) are recorded automatically when a sample is marked COLLECTED.
- Enter results for each test linked to an appointment; submitting a complete results set automatically marks the appointment and its sample as COMPLETED and pushes a real-time notification to connected reception and patient screens.

3.3.4 Administrator

- Manage the test catalogue: create, update, or remove tests (code, name, category, price, reference range, unit); the system prevents two tests from sharing the same test code.
- Manage report templates used to format printed/PDF results (header, footer, logo, notes).

- Manage user accounts, including activating/deactivating accounts.
- Review the live activity feed and the full audit log (who did what and when, across every module in this document).
- View all currently active sessions system-wide (user, IP address, browser, expiry) and forcibly revoke any session, which immediately signs that session out on its next request.

3.4 Description of Main Functions

Module	Main Functions
Authentication & User Management	Registration, login, logout, JWT session issuance, session-limit enforcement (max 3 concurrent), TOTP and e-mail two-factor authentication, profile and password management, user account administration.
Appointments	Patient self-booking, staff-assisted booking, appointment listing (patient and staff views), check-in with barcode generation, cancellation.
Sample & Test Management	Sample listing and status tracking, test catalogue management (create/update/delete tests).
Results & Reporting	Result entry and upsert, appointment/sample completion cascade, real-time status broadcast (Socket.io), public barcode-based report verification.
Invoicing	Invoice generation on booking, invoice/payment listing, payment collection with automatic PAID/PENDING status calculation.
Admin, Security & Audit	Report template management, system-wide audit log, live activity feed, active session monitoring and remote revocation.

Table 3.2 — Main functions grouped by module.

3.5 General Error Handling

The API returns a consistent JSON error shape with an HTTP status code and a human-readable message (and, for validation failures, a per-field error map) via a shared error-handling middleware (backend/src/middleware/error.js and utils/httpError.js). The table below explains what each status code means to an end user and what action to take.

HTTP Status	Typical Message	Meaning to the User	Recommended Action
400 Bad Request	"Validation failed" (with field-level detail) or a specific rule message, e.g. "Appointment date cannot be in the past."	One or more submitted fields did not meet the system's rules.	Correct the highlighted field(s) using the message shown next to each field and resubmit.
401 Unauthorized	"Invalid credentials or inactive account.", "Session has been revoked or expired.", "Invalid or expired authentication token."	Either the login details were wrong, the account is deactivated, or the current session is no longer valid.	Re-enter credentials, or log in again; if the account appears deactivated, contact an administrator.
403 Forbidden	"You are not allowed to access this resource."	The logged-in account's role does not have permission for this action.	Confirm the correct role is logged in; otherwise contact an administrator to request access.
404 Not Found	"Invoice not found.", "Sample not found.", "Appointment not found.", "No matching clinical	The referenced record does not exist or the identifier/barcode was entered incorrectly.	Re-check the identifier or barcode; if the record should exist, contact reception/admin.

HTTP Status	Typical Message	Meaning to the User	Recommended Action
	record found for this barcode."		
409 Conflict	"An account with this email already exists.", "A test with that code already exists."	The submitted value duplicates something already in the system.	Use a different e-mail address or test code, or edit the existing record instead of creating a new one.
500 Internal Server Error	Generic failure message (no sensitive detail returned to the client).	An unexpected server-side error occurred.	Retry the action; if it persists, report it to the technical/admin team with the time of the error so the server log and audit trail can be checked.

Table 3.3 — General error handling reference.

4. Critical Appraisal

4.1 Group Critical Appraisal

From a macro perspective, the group succeeded in delivering a working, role-segmented LIS covering the full patient journey: registration and authentication, appointment booking, sample collection, result entry, real-time status updates, invoicing/payment, and administrative oversight (audit logging and session security). The clearest evidence of a coherent overall design is that the six functional areas map cleanly onto six independent controller/route pairs sharing one PostgreSQL schema, which allowed the group to divide work along the same boundaries used for this appraisal.

Problems faced during development

- No automated test framework was configured for the backend (the package.json test script is a placeholder that simply exits with an error), so all testing documented in Section 1 had to be performed manually against a running instance, which was significantly more time-consuming than an automated suite and increased the risk of a manual step being skipped under time pressure.
- Coordinating six members around one shared PostgreSQL schema (database/schema.sql) created integration risk whenever more than one member needed to extend the same table (for example, the sessions table added later via a separate add_sessions_table.js migration script rather than the main schema file); this was managed by treating schema changes as reviewed, sequential migrations rather than direct edits.
- Testing (Section 1.3) surfaced two real defects that had not been caught during individual development: a missing lower-bound check on payment amounts in the Invoicing module, and an incorrect SQL parameter index in the Admin module's template-update query. Both illustrate the value of the cross-module, systematic testing pass described in Section 1, as opposed to relying only on each member's testing of their own code.

Possible future enhancements

- Introduce an automated test suite (e.g. Jest with Supertest against a test database) so that the manual test cases in Section 1 can be converted into a repeatable regression suite, catching defects like TC-23 and TC-27 automatically on every change.
- Add account lockout / rate-limiting after repeated failed login attempts, since the current login flow has no limit on password guesses beyond the general absence of brute-force protection.
- Extend validateBody()-style validation to the Invoicing and Admin modules, which currently perform ad hoc checks (or none, in the case of payment amounts) rather than using the shared validation rule engine already built for Authentication and other modules.
- Activate the SMS/e-mail notification preference fields already present in the patients table schema (notification_preference, sms_enabled, email_enabled) to automatically notify patients when results are ready, rather than requiring patients to check the portal themselves.
- Formalise sample and appointment status transitions as an explicit state machine, to prevent an invalid transition (e.g. marking a sample COMPLETED before it has been COLLECTED) that the current implementation does not explicitly guard against.

4.2 Individual Critical Appraisal

Each member's appraisal below focuses on the module they were individually responsible for, in line with the module ownership confirmed for this group. Names have been left as placeholders for each member to complete; the technical content reflects the actual implementation of that module and can be edited/extended by the responsible member before submission.

4.2.1 Authentication & User Management

Module responsibility: Authentication & User Management, comprising authController.js, middleware/auth.js, utils/jwt.js, utils/twoFactor.js and the users/user_profiles/user_sessions tables.

Technical challenges faced:

- Implementing two structurally different 2FA delivery paths (TOTP secret/QR generation via speakeasy vs. a time-boxed e-mail OTP) behind a single enableTwoFactor()/verifyTwoFactorLogin() interface, without duplicating the expiry and validation logic.
- Getting the session-limit and auto-revocation logic (createUserSession()) to behave correctly at exactly three active sessions, which required careful ordering of the count-check, revoke, and insert steps inside one function.
- Ensuring the JWT payload and the live user_sessions row stay consistent, so that revoking a session (logout, or admin-triggered revocation) is reflected immediately by authenticate() on the very next request.

Suggested individual enhancement:

- Add rate-limiting/lockout on the login endpoint after repeated failed password attempts for the same account or IP address.

4.2.2 Appointments

Module responsibility: Appointments, comprising appointmentController.js and the appointments/appointment_tests tables.

Technical challenges faced:

- Keeping the patient-initiated (createAppointment()) and staff-initiated (createAppointmentByStaff()) booking paths consistent, since both must create the same linked invoice and sample rows but differ in required fields and the identity of the booking party.
- Designing the cancelAppointment() authorization so that a patient can only cancel their own appointment while reception/admin can cancel any appointment, using a single function with role-dependent SQL rather than two near-duplicate functions.
- Making the past-date rejection in createAppointment() precise at the boundary of "now", so a booking made for the current minute is not incorrectly rejected.

Suggested individual enhancement:

- Prevent double-booking by validating that a patient does not already have another SCHEDULED appointment at an overlapping time.

4.2.3 Sample & Test Management

Module responsibility: Sample & Test Management, comprising sampleController.js, testController.js and the samples/tests tables.

Technical challenges faced:

- Designing updateSample() so that marking a sample COLLECTED also stamps collected_at/collected_by, while other status changes do not overwrite that history — this required branching the SQL rather than using one generic UPDATE.
- Handling the case where checkInAppointment() (in the Appointments module) is called before a sample row exists yet, so updateSample() and the check-in flow do not conflict when both can create a sample record.
- Preventing duplicate test codes in the shared test catalogue (createTest()) while still allowing every other field to be edited freely via updateTest().

Suggested individual enhancement:

- Introduce an explicit, enforced status state machine for samples (e.g. PENDING → COLLECTED → PROCESSING → COMPLETED) so an out-of-order transition is rejected rather than silently accepted.

4.2.4 Results & Reporting

Module responsibility: Results & Reporting, comprising resultController.js, services/socketService.js and the test_results table.

Technical challenges faced:

- Using an INSERT … ON CONFLICT DO UPDATE (upsert) in submitResults() so a technician can correct a previously entered result without a separate "edit" endpoint, while still recording who entered/edited it and when.
- Coordinating the three-way status cascade (test_results → appointments.status → samples.status) inside one transaction so results, appointment status, and sample status can never be left out of sync if part of the request fails.
- Wiring the real-time Socket.io broadcast so reception and patient screens update immediately, while ensuring a broadcast failure does not roll back the already-committed database changes.

Suggested individual enhancement:

- Add a review/approval step for a second technician or a pathologist before a submitted result is marked COMPLETED and released to the patient portal, rather than releasing immediately on entry.

4.2.5 Invoicing

Module responsibility: Invoicing, comprising invoiceController.js and the invoices/payments tables.

Technical challenges faced:

- Calculating payment_status by summing all rows in the payments table for an invoice, which needed care to correctly move an invoice from PENDING to PAID exactly when the cumulative total reaches (not exceeds) the net amount.
- During testing (Section 1.3, TC-23) it was found that collectPayment() does not reject a negative or zero amount_paid before writing a payment row, which is the single most significant defect surfaced across the whole system.
- Keeping invoice totals in sync with the tests selected at booking time (createAppointment() computes total_amount/net_amount from the tests table), given that test prices can later be edited by the Admin module.

Suggested individual enhancement:

- Fix the identified defect by validating amount_paid > 0 (and amount_paid <= outstanding balance) before inserting a payment row, and add this as a permanent regression test case.

4.2.6 Admin, Security & Audit

Module responsibility: Admin, Security & Audit, comprising adminController.js, services/auditService.js and the audit_logs/user_sessions/report_templates/tests tables.

Technical challenges faced:

- Building a system-wide audit trail (logAudit()) that every other module calls into, which required agreeing on a consistent action-naming and description convention across all six members' code so the audit log in Section 3.4.4 reads coherently.
- During testing (Section 1.3, TC-27) it was found that updateTemplate() binds its WHERE id = clause to the wrong SQL parameter placeholder ($2, already bound to header, instead of the id parameter), which can silently update the wrong report template.
- Implementing session listing and remote revocation (listSessions()/revokeSession()) so an administrator can see and end a session on another user's device without needing direct database access.

Suggested individual enhancement:

- Fix the identified defect by correcting the parameter index in the updateTemplate() query, and add an automated regression test asserting that only the targeted template row changes when multiple templates exist.

Appendix A: Full Test Logs

The table below records every test case executed for the LIS, in the format required: test case description (data values, module and function under test), test case result, and test case conclusion (including any action taken).

ID	Module / Function	Data Values Tested	Result	Conclusion / Action
TC-01	Auth — passwordStrength()	Password = "Abc1234" (7 chars)	**Pass**	Rejected with correct message; no action.
TC-02	Auth — passwordStrength()	Password = "Abcdefg1" (8 chars, all classes)	**Pass**	Accepted; no action.
TC-03	Auth — email()	Email = "not-an-email"	**Pass**	Rejected with correct message; no action.
TC-04	Auth — createUserSession()	4th login attempt with 3 sessions already active	**Pass**	Oldest session revoked automatically; no action.
TC-05	Auth — createUserSession()	2nd login attempt with 1 session already active	**Pass**	No revocation triggered, as expected; no action.
TC-06	Auth — enableTwoFactor() (TOTP)	Correct TOTP code from authenticator app	**Pass**	2FA enabled successfully; no action.
TC-07	Auth — verifyTwoFactorLogin() (Email OTP)	Code submitted 11 minutes after issue	**Pass**	Correctly rejected as expired; no action.
TC-08	Auth — verifyTwoFactorLogin() (Email OTP)	Code submitted 9 minutes after issue	**Pass**	Accepted; no action.
TC-09	Auth — authenticate() middleware	Request with revoked session jti	**Pass**	Rejected 401; no action.
TC-10	Appointments — createAppointment()	appointment_date = now − 1 minute	**Pass**	Rejected 400; no action.
TC-11	Appointments — createAppointment()	appointment_date = now + 1 minute, with 2 testIds	**Pass**	Appointment, invoice, and sample rows all created consistently; no action.
TC-12	Appointments — cancelAppointment()	Patient cancels own SCHEDULED appointment	**Pass**	Cancelled successfully; no action.
TC-13	Appointments — cancelAppointment()	Patient attempts to cancel another patient's appointment	**Pass**	Correctly returns 404 (no matching row for that patient); no action.
TC-14	Sample Mgmt — createTest()	testCode duplicates an existing test	**Pass**	Rejected 409; no action.
TC-15	Sample Mgmt — updateSample()	status = "COLLECTED", no existing sample row	**Pass**	New sample row created with barcode; no action.
TC-16	Sample Mgmt — updateSample()	status field omitted from request body	**Pass**	Rejected 400 "Status is required."; no action.
TC-17	Results — submitResults()	Valid results array, 2 tests, appointment exists	**Pass**	Results saved; appointment and sample both moved to COMPLETED; no action.

ID	Module / Function	Data Values Tested	Result	Conclusion / Action
TC-18	Results — submitResults()	appointmentId does not exist	**Pass**	Rejected 404; no action.
TC-19	Results — submitResults()	Array element missing resultValue	**Pass**	Rejected 400 with correct message; no action.
TC-20	Results — verifyReport()	Barcode does not match any sample	**Pass**	Rejected 404 with correct message; no action.
TC-21	Invoicing — collectPayment()	amount_paid = net_amount exactly	**Pass**	Invoice correctly moves to PAID; no action.
TC-22	Invoicing — collectPayment()	amount_paid < net_amount	**Pass**	Invoice correctly remains PENDING; no action.
TC-23	Invoicing — collectPayment()	amount_paid = −500	**Fail**	Negative amount accepted and summed into total, corrupting payment_status. Defect logged; assigned for fix (validate amount_paid > 0).
TC-23b	Invoicing — collectPayment() (regression, post-fix design)	amount_paid = −500 (after proposed fix)	**Not yet retested**	Pending code fix and re-test before final submission.
TC-24	Admin — createTest()	price omitted from request body	**Pass**	Rejected 400; no action.
TC-25	Admin — listSessions() / revokeSession()	Admin revokes another user's active session	**Pass**	Session revoked; subsequent request with that token rejected 401; no action.
TC-26	Admin — listAuditLogsFull()	Confirm audit entries appear for actions in TC-04, TC-12, TC-17, TC-25	**Pass**	All four actions correctly present in the audit log; no action.
TC-27	Admin — updateTemplate()	Update template B's name while template A also exists	**Fail**	Template A was updated instead of template B (WHERE clause bound to wrong parameter index). Defect logged; assigned for fix.
TC-27b	Admin — updateTemplate() (regression, post-fix design)	Repeat of TC-27 after correcting the parameter index	**Not yet retested**	Pending code fix and re-test before final submission.
TC-28	Security — authorizeRoles()	PATIENT token used against an ADMIN-only route	**Pass**	Rejected 403; no action.
TC-29	Security — authenticate()	Request with no Authorization header	**Pass**	Rejected 401; no action.
TC-30	Security — authenticate()	Malformed / tampered JWT	**Pass**	Rejected 401 "Invalid or expired authentication token."; no action.